

F1

Knowledge System

Web Semantics — Practical Assignment 2 — Report

Authors	Rodrigo Abreu (113626), Hugo Ribeiro (113402)
Course	Web Semantics
Year	2025–2026
Technology	Django 6 · GraphDB · OWL 2 DL · SPIN · RDFa · MF2

June 1, 2026

Contents

1	Introduction	3
1.1	Dataset	3
2	Ontology Definition (RDFS and OWL)	3
2.1	OWL Profile	3
2.2	Class Hierarchy	4
2.3	Base Class Inference Without Explicit <code>rdf:type</code>	4
2.3.1	<code>rdfs:domain</code> (for GraphDB)	4
2.3.2	<code>owl:equivalentClass</code> (for Protégé / HermiT)	4
2.4	Why <code>rdfs:domain</code> is Omitted on Foreign-Key Properties	5
2.5	Key OWL Axioms	6
2.6	Restriction Axioms for Inferred Subclasses	7
2.7	Inference Verification	7
2.8	Protégé Validation with HermiT Reasoner	7
2.8.1	OWL 2 Datatype Map Compliance	7
2.8.2	Design Decision	8
2.8.3	HermiT Verification Results	8
3	Inference Rules (SPIN)	9
3.1	Championship Inference	9
3.2	Teammate Inference	10
3.3	Classification Rules	10
3.4	RDF Reification	11
4	New Operations on the Data (SPARQL)	12
4.1	World Champions Who Raced as Teammates	12
4.2	Constructors That Fielded the Most World Champions	13
4.3	SPARQL Property Paths via <code>rdfs:subPropertyOf</code>	13
4.4	Reification Query: Championship Final Points	13
4.5	Veteran World Champions (Class Intersection)	14
4.6	Two-Hop Teammate Graph Traversal	14
4.7	Inference Used Directly in Application Views	14
4.7.1	Driver Statistics via <code>f1:wonRace</code> and <code>f1:PodiumResult</code>	14
4.7.2	Constructor Pilots via <code>f1:hadDriver</code>	15
4.7.3	Circuit Classification from <code>f1:ActiveCircuit</code> and <code>f1:HistoricCircuit</code>	15
4.7.4	Race History and Top Winners via <code>f1:heldAt</code> and <code>f1:wonRace</code>	16
5	Use and Integration of Wikidata and DBpedia	16
5.1	Driver Enrichment (Wikidata)	16
5.2	Constructor Enrichment (Wikidata)	17
5.3	Circuit Enrichment (DBpedia)	18
6	Publication of Semantic Data (RDFa and Microformats)	18
6.1	RDFa 1.1 Lite	19
6.2	Microformats 2	19
6.3	Why Both Standards?	20

6.4	Microformats 2 Parser Demonstration	20
6.5	RDFA Validation	20
7	Application Functionalities (UI)	21
7.1	New Pages Driven by Inferred Knowledge	21
7.2	Semantic Tools Page (/tools/)	21
7.3	Hero Section Enrichment	21
7.4	Navigation with Dropdown Menus	21
7.5	Card Hover Expansion	22
7.6	Deep Links	22
7.7	Inference Surfaced Throughout the Application	22
7.8	Podium Visibility Control	23
8	Conclusions	23
8.1	Key Achievements	24
8.2	Challenges and Solutions	24
9	Configuration to Run the Application	25
9.1	Prerequisites	25
9.2	Step-by-Step Setup	25
9.3	Protégé Validation	25
9.4	Re-running Inference	26
9.5	Data Files Summary	26

1. Introduction

This report describes **Practical Assignment 2** (TP2), a continuation of the F1 Knowledge Graph Explorer built in TP1. TP1 established the foundation: a Django web application backed by GraphDB, with all domain data stored as RDF triples and queried through SPARQL. It achieved a grade of **15/20**.

TP2 adds a full **semantic knowledge layer** on top of that foundation. The key additions are:

- An **OWL 2 DL ontology** that formally describes the F1 domain, enabling automatic entity classification directly from the data properties of each entity — without any explicit `rdf:type` assertions in the facts file.
- **15 SPIN rules** (SPARQL INSERT WHERE) that materialise inferences OWL cannot express: World Champions, Veterans, Historic Circuits, podium results, and the teammate relation between drivers who raced together.
- **RDF reification** annotating every championship-winning triple with the driver's final points total.
- **Live semantic enrichment** from Wikidata and DBpedia via SPARQLWrapper: driver death dates, team logos, circuit lap records, and more.
- **RDFa 1.1 Lite** (Schema.org) and **Microformats 2** embedded in every detail page, making the site machine-readable by search engines and IndieWeb tools alike.
- New application pages driven entirely by inferred knowledge: World Champions, Multi-Champions, Veterans, and Constructor Champions.

1.1. Dataset

The dataset is the **Ergast Formula 1 World Championship** (1950–2024): 75 seasons, 861 drivers, 212 constructors, 77 circuits, and approximately 25,000 race results. The raw data comes from 13 CSV files loaded into a GraphDB repository named `ws-formula1`.

2. Ontology Definition (RDFS and OWL)

The ontology is defined in `data/rdf/formula1_ontology.ttl` as an OWL 2 DL Turtle file. It is the **only hand-authored source file** in the data pipeline; all other RDF files are generated by scripts.

2.1. OWL Profile

The ontology uses **OWL 2 DL** — a decidable fragment of first-order logic. GraphDB is configured with the **OWL-Max with RDFS** ruleset, which applies `rdfs:domain`, `rdfs:range`, `rdfs:subClassOf`, `owl:inverseOf`, and `owl:SymmetricProperty` entailments at triple load time, before any SPARQL queries run.

2.2. Class Hierarchy

The ontology defines 18 OWL classes organised in a deep hierarchy:

```
f1:Entity      rdfs:subClassOf owl:Thing .
f1:Person      rdfs:subClassOf f1:Entity .
f1:Team        rdfs:subClassOf f1:Entity .
f1:Venue       rdfs:subClassOf f1:Entity .
f1:Driver      rdfs:subClassOf f1:Person .
f1:Constructor rdfs:subClassOf f1:Team .
f1:Circuit     rdfs:subClassOf f1:Venue .
f1:Race        rdfs:subClassOf f1:Event .
f1:Season      rdfs:subClassOf f1:Event .
f1:Result      rdfs:subClassOf f1:PerformanceRecord .
% Inferred subclasses (populated by SPIN rules):
f1:WorldChampion rdfs:subClassOf f1:Driver .
f1:MultichampionDriver rdfs:subClassOf f1:WorldChampion .
f1:Veteran       rdfs:subClassOf f1:Driver .
f1:ConstructorChampion rdfs:subClassOf f1:Constructor .
f1:ActiveCircuit rdfs:subClassOf f1:Circuit .
f1:HistoricCircuit rdfs:subClassOf f1:Circuit .
f1:PodiumResult  rdfs:subClassOf f1:Result .
```

Listing 1: Top-level class hierarchy in the ontology

2.3. Base Class Inference Without Explicit `rdf:type`

A central design requirement was that the facts file should contain **no explicit** `rdf:type` `f1:Driver` (or `Constructor`, `Circuit`) assertions. The ontology must infer these types purely from the properties present on each entity.

This is achieved using two complementary mechanisms:

2.3.1. *rdfs:domain* (for *GraphDB*)

Each “signature” property that appears exclusively on one entity class is declared with `rdfs:domain`. *GraphDB*’s OWL-Max ruleset then infers `rdf:type` at load time from any occurrence of that property:

```
f1:driverRef      rdfs:domain f1:Driver .
f1:constructorRef rdfs:domain f1:Constructor .
f1:circuitRef     rdfs:domain f1:Circuit .
```

Listing 2: `rdfs:domain` declarations as inference anchors

So `<driver/1> f1:driverRef "hamilton"` causes *GraphDB* to automatically infer `<driver/1> rdf:type f1:Driver` without any explicit assertion.

2.3.2. *owl:equivalentClass* (for *Protégé* / *HermiT*)

For validation in *Protégé*, the OWL reasoner *HermiT* requires a *necessary and sufficient* condition. Each base class is therefore defined with `owl:equivalentClass` using a `someValuesFrom` restriction:

```
f1:Driver owl:equivalentClass [  
  rdf:type owl:Restriction ;  
  owl:onProperty f1:driverRef ;  
  owl:someValuesFrom xsd:string  
] .
```

Listing 3: Bidirectional equivalentClass definition for Driver

This means: an individual is a `f1:Driver` **if and only if** it has at least one `f1:driverRef` value. Hermit can classify anonymous individuals without an explicit type assertion, making the ontology verifiable in Protégé.

Verification: Loading the facts file alone into GraphDB (without the ontology) returns 0 results for `SELECT * WHERE {?d rdf:type f1:Driver}`. After loading the integrated file (ontology + facts), the same query returns 861 drivers. The ontology, not the data, does the classification.

2.4. Why `rdfs:domain` is Omitted on Foreign-Key Properties

Properties like `f1:driverId`, `f1:constructorId`, and `f1:raceId` appear as **literal columns on multiple entity types**. For example, a `Result` row carries both a `f1:driverId` and a `f1:constructorId` literal. Declaring `rdfs:domain f1:Driver` on `f1:driverId` would cause GraphDB to infer that every `Result` individual is also a `Driver`, immediately triggering the `owl:disjointWith f1:Constructor` inconsistency. Domain declarations are intentionally omitted for these FK properties.

2.5. Key OWL Axioms

Construct	Applied to
<code>owl:FunctionalProperty</code>	race, driver, status, circuit, season, heldAt
<code>owl:SymmetricProperty</code>	wasTeammate
<code>owl:inverseOf</code>	hadDriver $\leftrightarrow$ droveFor
<code>rdfs:subPropertyOf</code>	wonRace $\sqsubseteq$ competedIn, achievedPodium $\sqsubseteq$ competedIn

2.6. Restriction Axioms for Inferred Subclasses

Three inferred subclasses carry `rdfs:subClassOf` restriction axioms that document the structural intent — even though the actual classification is performed by SPIN rules (Section 3):

```
f1:WorldChampion rdfs:subClassOf [
  rdf:type          owl:Restriction ;
  owl:onProperty   f1:wonChampionship ;
  owl:someValuesFrom f1:Season
] .
```

Listing 4: Restriction on WorldChampion: must have at least one wonChampionship

2.7. Inference Verification

After loading the integrated file and running the 15 SPIN rules, all 15 inference checks pass:

Mechanism	Inferred	Count
<code>rdfs:domain driverRef</code>	<code>f1:Driver</code> entities	861
<code>rdfs:domain constructorRef</code>	<code>f1:Constructor</code> entities	212
<code>rdfs:domain circuitRef</code>	<code>f1:Circuit</code> entities	77
SPIN <code>classify_WorldChampion</code>	<code>f1:WorldChampion</code>	34
SPIN <code>classify_MultichampionDriver</code>	<code>f1:MultichampionDriver</code>	17
SPIN <code>classify_Veteran</code>	<code>f1:Veteran</code>	86
SPIN <code>classify_ConstructorChampion</code>	<code>f1:ConstructorChampion</code>	17
SPIN <code>classify_ActiveCircuit</code>	<code>f1:ActiveCircuit</code>	24
SPIN <code>classify_HistoricCircuit</code>	<code>f1:HistoricCircuit</code>	45
SPIN <code>classify_PodiumResult</code>	<code>f1:PodiumResult</code>	3,397
SPIN <code>infer_wonRace</code>	<code>f1:wonRace</code> triples	1,128
SPIN <code>infer_wasTeammate</code>	<code>f1:wasTeammate</code> triples	10,012
SPIN <code>infer_wonChampionship</code>	<code>f1:wonChampionship</code> triples	75
<code>owl:inverseOf</code>	<code>f1:hadDriver</code> triples	2,150
SPIN <code>infer_heldAt</code>	<code>f1:heldAt</code> triples	1,125

2.8. Protégé Validation with HermiT Reasoner

The ontology was formally validated using the **HermiT 1.4.3** DL reasoner (bundled with Protégé 5.6.9) running programmatically via the `owlready2` Python library. This section documents the complete analysis, including a real issue encountered and the design decision taken to resolve it.

2.8.1. OWL 2 Datatype Map Compliance

During validation, HermiT raised an `UnsupportedDatatypeException` for three `rdfs:range` declarations:

```
f1:dob      rdfs:range xsd:date .    -- NOT in OWL 2 datatype map
f1:date     rdfs:range xsd:date .    -- NOT in OWL 2 datatype map
f1:year     rdfs:range xsd:gYear .   -- NOT in OWL 2 datatype map
```

Listing 5: Original range declarations causing HermiT rejection

The OWL 2 specification defines a strict *datatype map* — a subset of XML Schema datatypes that OWL 2 DL reasoners are required to support. This map includes `xsd:dateTime` and `xsd:dateTimeStamp` but **not** `xsd:date` or `xsd:gYear`. The teacher’s slides state that “A OWL faz uso dos tipos de dados definidos no XML Schema” (OWL uses XML Schema datatypes), but the key constraint is that only a *subset* of XML Schema types are in the OWL 2 DL datatype map.

2.8.2. Design Decision

The `rdfs:range` declaration is defined in RDFS as *optional*: it states what kind of values a property *may* take, but its absence does not prevent the property from functioning. The decision was to **remove the three non-compliant range declarations** rather than replacing them with an incorrect type (e.g., `xsd:dateTime`) or a loss of type information (e.g., `xsd:string`).

Justification: The semantic power of the ontology is entirely unchanged. All inference mechanisms — `rdfs:domain` for base class classification, `owl:equivalentClass` for Driver/Constructor/Circuit, `owl:disjointWith` for consistency checking, `owl:FunctionalProperty`, `owl:SymmetricProperty`, `owl:inverseOf`, and `rdfs:subPropertyOf` — are independent of these three range declarations. The domain declarations (`f1:driver rdfs:domain f1:Driver`, `f1:year rdfs:domain f1:Event`, `f1:date rdfs:domain f1:Race`) are preserved and still provide useful type inference. The actual data values in GraphDB remain typed as `xsd:date` and `xsd:gYear`, which GraphDB handles correctly under its OWL-Max ruleset.

2.8.3. HermiT Verification Results

After removing the three non-compliant range declarations, HermiT was run on the complete ontology (352 triples, 27 classes, 15 object properties, 42 datatype properties, 0 asserted individuals). The reasoning completed in **838 ms**.

Check	Result
Consistency	CONSISTENT — no exception, no contradiction
Unsatisfiable classes	0 — every named class has a non-empty extension
Equivalent class definitions	Driver $\equiv \exists \text{driverRef.string}$, Constructor $\equiv \exists \text{constructorRef.string}$, Circuit $\equiv \exists \text{circuitRef.string}$
Property hierarchy	wonRace $\sqsubseteq$ competedIn, achievedPodium $\sqsubseteq$ competedIn
Functional properties	circuit, driver, heldAt, race, season, status
Symmetric properties	wasTeammate
Disjoint classes	7 pairs verified (Driver $\perp$ Constructor, Driver $\perp$ Circuit, Circuit $\perp$ Constructor, Race $\perp$ Season, Driver $\perp$ Race, Result $\perp$ QualifyingResult, LapTime $\perp$ Result)

The full inferred class hierarchy produced by HermiT:

Listing 6: Class hierarchy inferred by HermiT from owl:equivalentClass + rdfs:subClassOf axioms

```

owl:Thing
  Entity
    Person  -> Driver  -> WorldChampion  -> MultichampionDriver
              -> Veteran
    Team    -> Constructor  -> ConstructorChampion
    Venue   -> Circuit      -> ActiveCircuit
              -> HistoricCircuit

  Event
    Race
    Season
  PerformanceRecord
    Result  -> PodiumResult
    QualifyingResult
    LapTime
    PitStop
    ConstructorResult
  Standing
    DriverStanding
    ConstructorStanding
  Status

```

This hierarchy proves the ontology’s inference power: the `WorldChampion`, `Veteran`, `ConstructorChampion`, `ActiveCircuit`, `HistoricCircuit`, and `PodiumResult` subclasses are all correctly positioned by HermiT using `rdfs:subClassOf` alone, confirming that the ontology structure is semantically sound.

3. Inference Rules (SPIN)

SPIN (SPARQL Inference Notation) fills the expressivity gap between OWL DL and the kind of reasoning the F1 domain requires. OWL DL — being based on Description Logic — has no mechanism for:

- **Aggregation:** COUNT, GROUP BY, MAX
- **Arithmetic comparisons** on data values: `positionOrder ≤ 3`
- **Negation-as-failure:** FILTER NOT EXISTS over derived patterns
- **Cross-entity joins:** same race, same constructor, different driver

The 15 rules are defined in `championship/spin_rules.py` as a list of Python dictionaries, each containing a `name`, `description`, and `query` (a SPARQL 1.1 INSERT WHERE statement). They are executed sequentially by the management command `python manage.py run_spin_rules`.

3.1. Championship Inference

The most important pair of rules infers the `f1:wonChampionship` property by identifying the driver who held position 1 in the `DriverStanding` at the *final round* of each season. OWL cannot express this because “final round” requires computing the maximum over a GROUP BY aggregate:

```

INSERT { ?driver f1:wonChampionship ?season . }
WHERE {
  ?ds f1:driverStandingsId ?anyId ;
    f1:race ?race ;
    f1:driver ?driver ;
    f1:position 1 .
  ?race f1:round ?round ;
    f1:year ?yr .
  ?season rdf:type f1:Season ;    -- type guard: prevents matching
    Race entities
    f1:year ?yr .

  {
    SELECT ?yr (MAX(?r) AS ?maxRound)
    WHERE { ?rc f1:round ?r ; f1:year ?yr . }
    GROUP BY ?yr
  }
  FILTER(?round = ?maxRound)
  FILTER NOT EXISTS { ?driver f1:wonChampionship ?season }
}

```

Listing 7: SPIN rule: infer_wonChampionship

Bug found and fixed: The initial version lacked the `?season rdf:type f1:Season` type guard. Because `Race` entities also carry `f1:year`, the variable `?season` matched all 17 races per season in addition to the single `Season` resource, producing $17 \times 75 = 1,275$ triples instead of 75. Adding the type guard corrected the count to exactly 75 (one per F1 season from 1950 to 2024). The same bug affected `infer_wonConstructorChampionship` and was fixed identically.

3.2. Teammate Inference

The `infer_wasTeammate` rule establishes 10,012 symmetric pairs between drivers who shared a constructor in the same race. This is a three-entity cross-join that OWL cannot express:

```

INSERT { ?d1 f1:wasTeammate ?d2 . ?d2 f1:wasTeammate ?d1 . }
WHERE {
  ?r1 f1:resultId ?id1 ; f1:race ?race ; f1:constructor ?ctor ;
    f1:driver ?d1 .
  ?r2 f1:resultId ?id2 ; f1:race ?race ; f1:constructor ?ctor ;
    f1:driver ?d2 .
  FILTER(?d1 != ?d2)
  FILTER NOT EXISTS { ?d1 f1:wasTeammate ?d2 }
}

```

Listing 8: SPIN rule: infer_wasTeammate

Note that the rule inserts *both* directions in the same `INSERT`, rather than relying on the `owl:SymmetricProperty` entailment, to ensure full materialisation in the graph.

3.3. Classification Rules

The following three classification rules illustrate the arithmetic and aggregation patterns that OWL cannot handle:

```

INSERT { ?driver rdf:type f1:MultichampionDriver . }
WHERE {
  SELECT ?driver (COUNT(?season) AS ?titles)
  WHERE { ?driver f1:wonChampionship ?season . }
  GROUP BY ?driver
  HAVING(?titles >= 2)
}

```

Listing 9: classify_MultichampionDriver: COUNT aggregation

```

INSERT { ?driver rdf:type f1:Veteran . }
WHERE {
  SELECT ?driver (COUNT(?r) AS ?entries)
  WHERE { ?r f1:resultId ?id ; f1:driver ?driver . }
  GROUP BY ?driver
  HAVING(?entries >= 100)
}

```

Listing 10: classify_Veteran: COUNT results ≥ 100

```

INSERT { ?circuit rdf:type f1:HistoricCircuit . }
WHERE {
  ?race f1:circuit ?circuit ; f1:year ?yr .
  {
    SELECT (MAX(?y) AS ?maxYear) WHERE { ?r f1:year ?y . }
  }
  FILTER NOT EXISTS {
    ?recentRace f1:circuit ?circuit ;
      f1:year ?recentYr .
    FILTER(xsd:integer(str(?recentYr)) > xsd:integer(str(?maxYear)) -
      10)
  }
}

```

Listing 11: classify_HistoricCircuit: MAX(year) arithmetic + negation

3.4. RDF Reification

The `reify_wonChampionship` rule creates **75 `rdf:Statement` nodes** — one per F1 World Drivers' Championship from 1950 to 2024. Each node annotates the existing `driver wonChampionship season` triple with the driver's final points total using the custom property `f1:finalPoints`:

```

INSERT {
  _:stmt rdf:type      rdf:Statement ;
         rdf:subject   ?driver ;
         rdf:predicate  f1:wonChampionship ;
         rdf:object     ?season ;
         f1:finalPoints ?pts .
}
WHERE {
  ?driver f1:wonChampionship ?season .
  ?season rdf:type f1:Season ; f1:year ?yr .
  ?ds rdf:type f1:DriverStanding ; -- type guard: prevents matching
    Result entities
}

```

```

    f1:driver ?driver ;
    f1:race ?race ;
    f1:points ?pts .
  ?race f1:round ?round ; f1:year ?yr .
  {
    SELECT ?yr (MAX(?r) AS ?maxRound)
    WHERE { ?rc f1:round ?r ; f1:year ?yr . }
    GROUP BY ?yr
  }
  FILTER(?round = ?maxRound)
  FILTER NOT EXISTS {
    ?s rdf:type rdf:Statement ;
      rdf:subject ?driver ; rdf:predicate f1:wonChampionship ;
      rdf:object ?season
  }
}

```

Listing 12: SPIN rule: reify__wonChampionship

Why reification? The triple `driver wonChampionship season` already exists as a plain triple. Reification allows attaching metadata *to that triple itself* — not to the driver (who has many seasons) or the season (which has one champion). This is the canonical RDF use case: making statements about statements. The `f1:finalPoints` property is declared in the ontology with `rdfs:domain rdf:Statement`, ensuring it only appears on reification nodes.

4. New Operations on the Data (SPARQL)

This section presents SPARQL queries that are **only possible because of inferred knowledge** — none can be answered from the raw CSV data directly.

4.1. World Champions Who Raced as Teammates

Combines `f1:wasTeammate` (SPIN-inferred) with `rdf:type f1:WorldChampion` (SPIN-inferred). Both properties are absent from the raw data:

```

SELECT ?aLabel ?bLabel WHERE {
  ?a rdf:type f1:WorldChampion ; rdfs:label ?aLabel ; f1:wasTeammate
    ?b .
  ?b rdf:type f1:WorldChampion ; rdfs:label ?bLabel .
  FILTER(?a != ?b)
  FILTER(STR(?aLabel) < STR(?bLabel))
}
ORDER BY ?aLabel

```

Listing 13: Champions who shared a constructor in the same race

Result: Alain Prost & Ayrton Senna, Prost & Niki Lauda, Prost & Nigel Mansell (the legendary McLaren era confirmed by pure graph traversal).

4.2. Constructors That Fielded the Most World Champions

Uses `f1:hadDriver` (OWL `owl:inverseOf`-inferred from `f1:drovFor`) combined with `rdf:type f1:WorldChampion`:

```
SELECT ?ctorLabel (COUNT(DISTINCT ?d) AS ?champions) WHERE {
  ?ctor f1:hadDriver ?d .           -- inverseOf f1:drovFor, not in
    raw data
  ?d    rdf:type f1:WorldChampion .
  ?ctor rdfs:label ?ctorLabel .
}
GROUP BY ?ctor ?ctorLabel
ORDER BY DESC(?champions)
LIMIT 8
```

Listing 14: Constructor champion headcount via `inverseOf`-inferred `hadDriver`

Result: McLaren (15), Ferrari (15), Williams (11), Team Lotus (10).

4.3. SPARQL Property Paths via `rdfs:subPropertyOf`

The ontology declares `wonRace` $\sqsubseteq$ `competedIn` and `achievedPodium` $\sqsubseteq$ `competedIn`. This enables alternation path queries that would otherwise require a `UNION`:

```
SELECT DISTINCT ?cLabel WHERE {
  ?h rdfs:label "Lewis Hamilton" .
  ?h (f1:wonRace | f1:achievedPodium) ?race .
  ?race f1:heldAt ?circuit ; rdfs:label ?cLabel .
}
```

Listing 15: Circuits where Hamilton won or achieved a podium

4.4. Reification Query: Championship Final Points

The only way to retrieve the final championship points for each winner is through the reified `rdf:Statement` nodes created by the `reify_wonChampionship` SPIN rule:

```
SELECT ?driverLabel ?yr ?pts WHERE {
  ?stmt rdf:type      rdf:Statement ;
        rdf:predicate f1:wonChampionship ;
        rdf:subject   ?driver ;
        rdf:object     ?season ;
        f1:finalPoints ?pts .
  ?driver rdfs:label ?driverLabel .
  ?season f1:year     ?yr .
}
ORDER BY DESC(?yr)
```

Listing 16: Championship points retrieved from `rdf:Statement` reification nodes

Sample results: Lewis Hamilton 2020 (347 pts), Max Verstappen 2023 (575 pts), Michael Schumacher 2004 (148 pts).

4.5. Veteran World Champions (Class Intersection)

OWL class intersection of two independently SPIN-inferred classes:

```
SELECT ?dLabel WHERE {
  ?d rdf:type f1:WorldChampion ;
    rdf:type f1:Veteran ;           -- 100+ race entries, independent
    SPIN rule
    rdfs:label ?dLabel .
}
ORDER BY ?dLabel
```

Listing 17: Drivers classified as both WorldChampion and Veteran

4.6. Two-Hop Teammate Graph Traversal

The `owl:SymmetricProperty` on `wasTeammate` enables arbitrary-length path queries. This finds all drivers who were teammates of Hamilton's own teammates:

```
SELECT DISTINCT ?tLabel WHERE {
  ?h rdfs:label "Lewis Hamilton" .
  ?h f1:wasTeammate/f1:wasTeammate ?t . -- path operator
  ?t rdfs:label ?tLabel .
  FILTER(?t != ?h)
}
```

Listing 18: Teammates of Hamilton's teammates (2-hop traversal)

4.7. Inference Used Directly in Application Views

A critical principle of semantic web applications is that the application should **query what the reasoning engine already knows**, not recompute it in application code. The following queries demonstrate how each inferred property and class is used directly in the Django views to power the web pages.

4.7.1. Driver Statistics via `f1:wonRace` and `f1:PodiumResult`

The driver detail page previously counted wins by scanning all results and checking `positionOrder = 1` in Python. It now queries the SPIN-inferred `f1:wonRace` property and the `f1:PodiumResult` class directly:

```
SELECT (COUNT(DISTINCT ?race) AS ?n) WHERE {
  <res:driver/1> f1:wonRace ?race .
}
```

Listing 19: Win count from SPIN-inferred `f1:wonRace` (driver detail view)

```
SELECT ?pos WHERE {
  ?r rdf:type f1:PodiumResult ;
    f1:driver <res:driver/1> ;
    f1:positionOrder ?pos .
}
```

Listing 20: Podium breakdown from SPIN-inferred `f1:PodiumResult`

The championship count displayed in the stats uses `f1:wonChampionship`:

```
SELECT (COUNT(DISTINCT ?season) AS ?n) WHERE {
  <res:driver/1> f1:wonChampionship ?season .
}
```

Listing 21: Championship count from SPIN-inferred `f1:wonChampionship`

4.7.2. Constructor Pilots via `f1:hadDriver`

The constructor detail page previously joined results to enumerate all drivers who raced for a team. It now uses the OWL `owl:inverseOf`-inferred `f1:hadDriver` as the primary driver source, with `f1:wonRace` and `f1:PodiumResult` for per-driver statistics:

```
SELECT ?driver ?driverLabel
  (COUNT(DISTINCT ?r) AS ?races)
  (COUNT(DISTINCT ?winRace) AS ?wins)
  (COUNT(DISTINCT ?pr) AS ?podiums)
  (MIN(?year) AS ?firstYear) (MAX(?year) AS ?lastYear)
WHERE {
  <res:constructor/6> f1:hadDriver ?driver .      -- OWL inverseOf
  ?driver rdfs:label ?driverLabel .
  ?r f1:resultId ?anyId ;
    f1:constructor <res:constructor/6> ;
    f1:driver ?driver ; f1:race ?race .
  ?race f1:year ?year .
  OPTIONAL {
    ?driver f1:wonRace ?winRace .                -- SPIN-inferred
    ?r2 f1:resultId ?wrid ; f1:race ?winRace ;
      f1:driver ?driver ; f1:constructor <res:constructor/6> .
  }
  OPTIONAL {
    ?pr rdf:type f1:PodiumResult ;                -- SPIN-inferred
    f1:driver ?driver ;
    f1:constructor <res:constructor/6> ; f1:race ?race .
  }
}
GROUP BY ?driver ?driverLabel
ORDER BY DESC(?wins) DESC(?races)
```

Listing 22: Driver roster via OWL-inferred `f1:hadDriver` + SPIN-inferred `f1:wonRace`

4.7.3. Circuit Classification from `f1:ActiveCircuit` and `f1:HistoricCircuit`

The most important design change: the application previously recomputed circuit activity status in Python by comparing `MAX(year)` across all races in the dataset. It now queries the SPIN-inferred types directly:

```
-- Was: Python max_year = max(last_year for ...) ; active = last_year
== max_year
-- Now: query what the reasoning engine already knows

ASK WHERE { <res:circuit/6> rdf:type f1:ActiveCircuit }
ASK WHERE { <res:circuit/6> rdf:type f1:HistoricCircuit }

-- For the circuits list: fetch both sets at once
```

```
SELECT ?uri WHERE { ?uri rdf:type f1:ActiveCircuit }
SELECT ?uri WHERE { ?uri rdf:type f1:HistoricCircuit }
```

Listing 23: Circuit active/historic status from SPIN-inferred rdf:type

The circuits list page now offers three filter states: *Active* (24 circuits, `f1:ActiveCircuit`), *Historic* (45 circuits, `f1:HistoricCircuit`), and *Inactive*. The Active and Historic filters are driven entirely by SPIN inference; no application code recomputes the classification.

4.7.4. Race History and Top Winners via `f1:heldAt` and `f1:wonRace`

The circuit detail page race history previously used the raw `f1:circuit` FK property and joined results to find position-1 finishers. It now uses the SPIN-inferred `f1:heldAt` and `f1:wonRace`:

```
SELECT ?winnerLabel ?winnerId (COUNT(DISTINCT ?race) AS ?wins) WHERE {
  ?winner f1:wonRace ?race ;                -- SPIN-inferred
          rdfs:label ?winnerLabel ;
          f1:driverId ?winnerId .
  ?race   f1:heldAt <res:circuit/6> . -- SPIN-inferred
}
GROUP BY ?winner ?winnerLabel ?winnerId
ORDER BY DESC(?wins)
LIMIT 5
```

Listing 24: Top race winners at a circuit using SPIN-inferred `f1:wonRace` + `f1:heldAt`

Key principle: Each SPARQL query in this subsection replaces equivalent Python logic that recomputed information the reasoning engine had already materialised. Querying inferred data directly is both semantically correct and more efficient — the graph already holds the answer.

5. Use and Integration of Wikidata and DBpedia

External enrichment is fetched via **SPARQLWrapper** from the SPARQL endpoints of Wikidata (<https://query.wikidata.org/sparql>) and DBpedia (<https://dbpedia.org/sparql>). All calls are **deferred to after page render** — a JavaScript `fetch()` call triggers the enrichment after the page is visible, injecting data into the hero section asynchronously. This eliminates the 2–8 second SPARQL timeout from the critical page load path.

5.1. Driver Enrichment (Wikidata)

Wikidata is queried for F1 drivers using occupation property `wdt:P106` `wd:Q10843402` (Formula One driver), matched by label:

```
SELECT ?item ?birthPlaceLabel ?description ?image
       ?deathDate ?height ?fatherLabel
       (GROUP_CONCAT(DISTINCT ?childLabel; separator="|") AS
        ?childNames)
WHERE {
```

```

?item wdt:P106 wd:Q10843402 ;
      rdfs:label ?label .
OPTIONAL { ?item wdt:P19 ?birthPlace } -- place of birth
OPTIONAL { ?item wdt:P18 ?image }      -- portrait photo
OPTIONAL { ?item wdt:P570 ?deathDate } -- date of death
OPTIONAL { ?item wdt:P2048 ?height }   -- height in cm
OPTIONAL { ?item wdt:P22 ?father .
          ?father rdfs:label ?fatherLabel
          FILTER(LANG(?fatherLabel) = "en") }
OPTIONAL { ?item wdt:P40 ?child .
          ?child rdfs:label ?childLabel
          FILTER(LANG(?childLabel) = "en") }
OPTIONAL { ?item schema:description ?description
          FILTER(LANG(?description) = "en") }
SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
FILTER(LANG(?label) = "en")
FILTER(LCASE(STR(?label)) = "ayrton senna")
}
GROUP BY ?item ?birthPlaceLabel ?description ?image ?deathDate
         ?height ?fatherLabel
LIMIT 1

```

Listing 25: Wikidata query for driver enrichment with new fields

New fields added in TP2 (beyond TP1’s `birthPlace` and `image`):

Wikidata property	Field	Displayed as
P570	<code>deathDate</code>	† suffix on driver name; lifespan chip
P2048	<code>height</code>	Height chip in hero metadata
P22	<code>fatherName</code>	“Father: Graham Hill” (F1 dynasties)
P40	<code>childNames</code>	“Son: Mick Schumacher” (pipe-separated list)

The death date is particularly significant for historical drivers. Senna (1994), Ratzenberger (1994), Rindt (1970), Clark (1968), and others are now distinguished visually with the † symbol. The family links reveal well-known F1 dynasties: Graham/Damon Hill, Gilles/Jacques Villeneuve, Michael/Mick Schumacher.

5.2. Constructor Enrichment (Wikidata)

Constructors are queried as F1 racing teams (`wdt:P31 wd:Q41298738`):

```

SELECT ?item ?foundingDate ?hqLabel ?logo ?website
       ?countryLabel ?founderLabel ?dissolved WHERE {
?item wdt:P31 wd:Q41298738 ;
      rdfs:label ?label .
OPTIONAL { ?item wdt:P571 ?foundingDate } -- inception date
OPTIONAL { ?item wdt:P159 ?hq }          -- headquarters
OPTIONAL { ?item wdt:P154 ?logo }        -- official logo image
OPTIONAL { ?item wdt:P856 ?website }     -- official website
OPTIONAL { ?item wdt:P17 ?country }      -- country of origin
OPTIONAL { ?item wdt:P112 ?founder }     -- founded by
OPTIONAL { ?item wdt:P576 ?dissolved }   -- dissolved date
SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
FILTER(LANG(?label) = "en")
FILTER(LCASE(STR(?label)) = "ferrari")

```

```
}
LIMIT 1
```

Listing 26: Wikidata query for constructor enrichment

The team logo (P154) replaces the generic Wikipedia thumbnail on the constructor detail page. The `dissolved` date displays a “Dissolved YYYY” chip for defunct teams such as Tyrrell, Brabham, and Lotus.

5.3. Circuit Enrichment (DBpedia)

DBpedia provides track-specific facts not present in the Ergast dataset. A key finding during development was that F1 circuits in DBpedia are **not** classified as `dbo:RaceTrack` — they use generic types (`owl:Thing`, `geo:SpatialThing`). Label-matching queries with a type constraint consistently returned zero results.

The solution is to **construct the DBpedia resource URI directly** from the circuit name, bypassing the need for any label or type matching:

```
resource_uri = "http://dbpedia.org/resource/" + circuit_name.replace("
    ", "_")
# "Circuit de Monaco" --> dbr:Circuit_de_Monaco
```

Listing 27: Direct URI construction for DBpedia circuit lookup (external_data.py)

```
SELECT ?abstract ?length ?turns ?capacity ?opened ?lapRecord WHERE {
  OPTIONAL { <dbr:Circuit_de_Monaco> dbo:abstract ?abstract
             FILTER(LANG(?abstract) = "en") }
  OPTIONAL { <dbr:Circuit_de_Monaco> dbp:length ?length }
  OPTIONAL { <dbr:Circuit_de_Monaco> dbp:turns ?turns }
  OPTIONAL { <dbr:Circuit_de_Monaco> dbp:capacity ?capacity }
  OPTIONAL { <dbr:Circuit_de_Monaco> dbp:opened ?opened }
  OPTIONAL { <dbr:Circuit_de_Monaco> dbp:lapRecord ?lapRecord }
  FILTER(BOUND(?turns) || BOUND(?capacity) || BOUND(?abstract))
}
LIMIT 1
```

Listing 28: DBpedia query using direct resource URI

DBpedia property	Field	Monaco	Silverstone
dbp:turns	corners	12	6
dbp:capacity	spectators	37,000	164,000
dbp:opened	year built	1929	1948
dbp:lapRecord	fastest lap text	(varies)	(varies)

These facts are injected into the “Circuit details” list and a dedicated “Lap Record” panel on the circuit detail page.

6. Publication of Semantic Data (RDFa and Microformats)

Every detail page in the application simultaneously embeds two semantic markup standards. These serve different parsers and different use cases.

6.1. RDFa 1.1 Lite

RDFa 1.1 Lite uses the Schema.org vocabulary, declared with the `vocab` attribute. It is consumed by Google’s Knowledge Graph, search engine rich results, and the W3C pyRDFa distiller.

```
<article class="driver-hero h-card"
  vocab="https://schema.org/"
  typeof="Person"
  resource="{ request.build_absolute_uri }">
  <div class="driver-hero-name p-name" property="name">
    Lewis Hamilton
  </div>
  <span property="nationality">British</span>
  <time property="birthDate" datetime="1985-01-07">b. 1985-01-07</time>
  <link property="sameAs" href="https://en.wikipedia.org/wiki/
    Lewis_Hamilton" />
</article>
```

Listing 29: RDFa on the driver detail page

Page	typeof	Key properties
Driver detail	Person	name, birthDate, nationality, sameAs
Constructor detail	SportsOrganization	name, nationality, sameAs
Circuit detail	SportsActivityLocation	name, latitude, longitude, addressLocality
Race detail	SportsEvent	name, startDate, sameAs
Season detail	Event	name, startDate
Champions list	ItemList	itemListElement (each item: Person)

The `resource` attribute is set to `{ request.build_absolute_uri }`, giving each entity a canonical URI tied to its page URL.

6.2. Microformats 2

Microformats 2 requires no vocabulary declaration and is parsed by IndieWeb tools, social readers, and contact importers. It coexists with RDFa on the same element:

```
<article class="hero h-card h-adr"
  vocab="https://schema.org/"
  typeof="SportsActivityLocation"
  resource="{ request.build_absolute_uri }">
  <span class="p-name" property="name">Circuit de Monaco</span>
  <span class="p-locality" property="addressLocality">Monte Carlo</span>
  <span class="p-country-name" property="addressCountry">Monaco</span>
  <a class="u-url" property="sameAs"
    href="https://en.wikipedia.org/wiki/Circuit_de_Monaco">Wikipedia
  </a>
</article>
```

Listing 30: Microformats 2 on the circuit detail page (h-card + h-adr)

Class	Standard	Used on
h-card	hCard	Driver, Constructor, Circuit pages
h-event	hEvent	Race, Season pages
h-adr	hAdr	Circuit address block
p-name	hCard/hEvent	Entity name
p-nationality	hCard	Driver/Constructor nationality
p-country-name	hAdr	Circuit country
p-locality	hAdr	Circuit city
dt-bday	hCard	Driver date of birth
dt-start	hEvent	Race/Season start date
u-url	hCard/hEvent	Wikipedia link (<code>rdfs:seeAlso</code>)

6.3. Why Both Standards?

RDFa integrates with the OWL ontology namespace and is consumed by SPARQL-aware tools and Google’s Knowledge Graph. Microformats 2 requires no vocabulary declaration and is parsed by IndieWeb tools, social readers, and contact importers without configuration. Embedding both on the same HTML element demonstrates mastery of both standards and their respective trade-offs.

6.4. Microformats 2 Parser Demonstration

The `/tools/` page includes a live parser powered by `mf2py` (a Python Microformats 2 library). It fetches any page of the site and returns the extracted JSON, demonstrating the full round-trip:

Listing 31: Calling the live MF2 parser endpoint

```
GET /api/parse-microformats/?url=http://localhost:8000/drivers/1/

# Response (abbreviated):
{
  "items": [{
    "type": ["h-card"],
    "properties": {
      "name": ["Lewis Hamilton"],
      "nationality": ["British"],
      "bday": ["1985-01-07"],
      "url": ["https://en.wikipedia.org/wiki/Lewis_Hamilton"]
    }
  }]
}
```

6.5. RDFa Validation

The `/tools/` page provides direct links to three external validators:

- **W3C pyRDFa Distiller** — extracts RDFa as Turtle RDF from any page URL
- **Google Rich Results Test** — validates Schema.org structured data
- **RDFa Play** (<https://rdfa.info/play/>) — paste HTML, visualise the extracted RDF graph

7. Application Functionalities (UI)

This section describes the features added in TP2 that were not present in TP1.

7.1. New Pages Driven by Inferred Knowledge

Four new pages are **entirely powered by SPIN-inferred classes**. None of these pages could exist without the ontology and inference layer:

URL	Page	SPIN class	Count
/champions/	World Champions	f1:WorldChampion	34
/champions/multi/	Multi-Champions	f1:MultichampionDriver	17
/champions/veterans/	Veterans	f1:Veteran	86
/constructors/champions/	Constructor Champions	f1:ConstructorChampion	17

7.2. Semantic Tools Page (/tools/)

A dedicated page demonstrating three semantic web features:

1. **RDF Reification viewer** — executes a live SPARQL query over the `rdf:Statement` nodes, displaying championship points per winner per year.
2. **Microformats 2 parser** — accepts any page URL and displays the parsed MF2 JSON using the `mf2py` library.
3. **RDFa extraction links** — direct links to the W3C pyRDFa Distiller and Google Rich Results Test pre-loaded with the site's pages.

7.3. Hero Section Enrichment

All detail pages (driver, constructor, circuit) now display:

- A **description paragraph** sourced asynchronously from Wikidata or DBpedia, with a small source attribution link.
- A **clickable hero photo** that opens a full-size lightbox using the native HTML `<dialog>` element — no library required.
- For **drivers**: death date (†) displayed in the name, height chip, and family dynasty links (father / children who also raced in F1).
- For **constructors**: Wikidata team logo replaces the Wikipedia thumbnail, founder and official website injected into the stats list.
- For **circuits**: DBpedia facts (turns, capacity, year opened, lap record) injected into the Circuit details card.

7.4. Navigation with Dropdown Menus

The navigation bar was restructured with dropdown menus to organise the new pages:

- **Drivers** ▸ All Drivers / World Champions / Multi-Champions / Veterans

- **Constructors** ▷ All Constructors / Constructor Champions

7.5. Card Hover Expansion

Entity cards (drivers, constructors, circuits, champions) expand on hover using `transform: scale(1.08)` with `transform-origin: top center`. Because CSS transforms do not affect layout flow, adjacent cards are never pushed — the expanded card floats over its neighbours via `z-index: 10`.

7.6. Deep Links

Navigation shortcuts were added throughout the application:

- Constructor career chips in the driver detail page link to constructor detail pages.
- Constructor names in race results link to constructor detail pages.
- Circuit names in the season detail calendar link to circuit detail pages.
- Race winner constructors in the season calendar link to constructor detail pages.

7.7. Inference Surfaced Throughout the Application

A systematic audit identified that several SPIN-inferred properties and classes were being *computed in Python* rather than queried from the graph. These were corrected so that the application always queries what the reasoning engine already knows (documented in detail in Section 4.7).

Page	Before	After
Driver detail — wins	Python: <code>count positionOrder = 1</code>	SPARQL: <code>f1:wonRace COUNT</code>
Driver detail — podiums	Python: <code>count positionOrder ≤ 3</code>	SPARQL: <code>rdf:type f1:PodiumResult</code>
Driver detail — championships	Not shown	SPARQL: <code>f1:wonChampionship COUNT</code>
Constructor detail — pilots	Result join to enumerate drivers	SPARQL: <code>f1:hadDriver (owl:inverseOf)</code>
Constructor detail — wins	<code>SUM(IF(positionOrder=1,</code>	SPARQL: <code>f1:wonRace</code>
Constructor detail — championships	Not shown	SPARQL: <code>f1:wonConstructorChampionship</code>
Circuit detail — active status	Python: <code>last_year == MAX(year)</code>	SPARQL: <code>rdf:type f1:ActiveCircuit</code>
Circuit detail — historic status	Not distinguished from inactive	SPARQL: <code>rdf:type f1:HistoricCircuit</code>
Circuit detail — top winners	Python dict aggregation over race rows	SPARQL: <code>f1:wonRace + f1:heldAt</code>
Circuit detail — race history	Raw FK: <code>f1:circuit</code>	Semantic link: <code>f1:heldAt</code>
Circuits list — filter	Active / Inactive only (Python)	Active / Historic / Inactive (GraphDB)
All detail pages — top circuits	Raw FK: <code>f1:circuit</code>	Semantic link: <code>f1:heldAt</code>

The stats cards in driver and constructor detail pages now annotate each inferred value with its source property (e.g. “via `f1:wonRace`”, “via `f1:PodiumResult`”, “via `f1:hadDriver`”), making the inference layer visible to the user.

7.8. Podium Visibility Control

The “all-time wins” podium at the top of the Drivers, Constructors, and Circuits list pages is now hidden when browsing pages 2 and beyond. On page 1 it provides a featured highlight; on subsequent pages it would be confusing context, so it is suppressed with a `{if page == 1}` condition in the template.

8. Conclusions

TP2 successfully demonstrates the end-to-end lifecycle of a semantic web application, extending TP1’s data foundation with a full knowledge layer.

8.1. Key Achievements

- **OWL 2 DL ontology** with 18 classes, 15+ properties, and 7 disjointness axioms. Verified in both GraphDB (OWL-Max ruleset) and Protégé (HermiT 1.4.3, 838 ms).
- **15 SPIN rules** covering all classification and inference patterns that OWL DL cannot express. All 15 pass with verified real-world counts.
- **RDF reification** of 75 championship-winning triples with final points annotation — a canonical demonstration of making statements about statements.
- **Inference fully surfaced in the application:** after a systematic audit, every inferred property and class is now queried directly from the graph. Application code no longer recomputes what the reasoning engine already knows (`f1:wonRace`, `f1:PodiumResult`, `f1:hadDriver`, `f1:heldAt`, `f1:ActiveCircuit`, `f1:HistoricCircuit`, `f1:wonChampionship`, `f1:wonConstructorChampionship`).
- **Linked data integration** from two SPARQL endpoints (Wikidata, DBpedia) and the Wikipedia REST API, with each data source contributing distinct information not available from the others.
- **Dual semantic markup** (RDFa 1.1 Lite + Microformats 2) on all detail pages, serving both schema-aware and schema-agnostic parsers.
- **Four new application pages** driven entirely by inferred knowledge — pages that could not exist without the ontology.

8.2. Challenges and Solutions

- **Type guard omission in SPIN rules:** The `infer_wonChampionship` rule initially produced 1,200 triples instead of 75 because the `?season` variable matched `Race` entities (which also carry `f1:year`). Adding `?season rdf:type f1:Season` fixed the count.
- **DBpedia classification:** F1 circuits are not typed as `dbo:RaceTrack` in DBpedia. The solution was to construct the resource URI directly from the circuit name instead of using label-matching queries.
- **Reification entity confusion:** Both `DriverStanding` and `Result` entities carry `f1:driver` and `f1:points`, causing duplicate reification nodes. Adding `?ds rdf:type f1:DriverStanding` restricted the join to the correct entity type.
- **GraphDB ruleset and import method:** The `rdfsplus-optimized` ruleset does not materialise `rdfs:domain` inference during bulk REST API loads. Switching to `owl-max-optimized` and using GraphDB's server-side import API (the same pipeline as the Workbench UI) resolved the issue.
- **Inference generated but not consumed:** An audit revealed that several SPIN-inferred properties (`f1:wonRace`, `f1:heldAt`, `f1:hadDriver`, `f1:PodiumResult`) and classes (`f1:ActiveCircuit`, `f1:HistoricCircuit`) were being recomputed in Python instead of queried from the graph. The application was corrected to query the reasoning engine's output directly, eliminating redundant computation and making the inference layer the authoritative source for all derived data.

9. Configuration to Run the Application

9.1. Prerequisites

- Python 3.12+
- GraphDB Free running on <http://localhost:7200>, with a repository named **ws-formula1-owlmax** using the **owl-max-optimized** ruleset (created automatically by `scripts/create_graphdb_repo.sh`)
- Ergast CSV files in `data/raw/`

9.2. Step-by-Step Setup

Listing 32: Complete setup from a fresh clone

```
# 1. Virtual environment and dependencies
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# 2. Environment configuration
cp .env.example .env
# Edit .env: GRAPHDB_BASE_URL, GRAPHDB_REPOSITORY, DJANGO_SECRET_KEY

# 3. Generate RDF facts from CSV (no rdf:type for base classes)
python scripts/csv_to_rdf.py
# Output: data/rdf/formula1.nt (~798 MB)

# 4. Merge ontology + facts into one file (for GraphDB and Protege)
python scripts/merge_integrated.py
# Output: data/rdf/formula1_integrated.nt

# 5. Load into GraphDB
# Workbench > ws-formula1 > Import > Upload RDF file
# Select: data/rdf/formula1_integrated.nt
# (or: bash scripts/load_rdf_to_graphdb.sh)

# 6. Run all 15 SPIN inference rules (~1 min)
python manage.py run_spin_rules
# Expected output: 15/15 rules applied successfully.

# 7. Django setup (SQLite for auth only)
python manage.py migrate
python manage.py createsuperuser

# 8. Start the development server
python manage.py runserver
```

Open <http://localhost:8000>. Admin panel: <http://localhost:8000/admin-panel/login/>.

9.3. Protégé Validation

To validate the ontology in Protégé 5.6.9 with HermiT:

1. Open Protégé and load `data/rdf/formula1_integrated.nt` (File → Open from File, format: N-Triples).

2. Select Reasoner → HermiT 1.4.3.
3. Click Start Reasoner. HermiT completes in under 1 second on schema only (0 explicit individuals — all types are inferred from properties).
4. Navigate to the Entities tab → Classes to verify the inferred class hierarchy matches Section 2.8.
5. No unsatisfiable classes should appear (red background on any class would indicate a modelling error).

Note on xsd:date / xsd:gYear: The ontology intentionally omits `rdfs:range` declarations for `f1:dob`, `f1:date`, and `f1:year` because `xsd:date` and `xsd:gYear` are not in the OWL 2 datatype map (Section 2.8). The actual data values in GraphDB remain typed as `xsd:date/xsd:gYear` — only the ontology range assertions are omitted to maintain full OWL 2 DL compliance.

9.4. Re-running Inference

All SPIN rules are idempotent. To re-run after a data change:

```
python manage.py run_spin_rules
```

9.5. Data Files Summary

File	Purpose	In git?
data/rdf/formula1_ontology.ttl	OWL 2 DL ontology (hand-authored)	Yes
data/rdf/formula1.nt	Facts only, no <code>rdf:type</code> for base classes	No (generated)
data/rdf/formula1_integrated.nt	Ontology + facts merged (for Protégé)	No (generated)
championship/spin_rules.py	15 SPIN rules as Python module	Yes